

5. Project Development Phase

ComicCraft – AI Comic Story Creator Using Gemini Models

Implementing the AI generation pipeline, backend, and frontend

1. Development Overview

ComicCraft is implemented as a web application using FastAPI, Jinja2 templates, Gemini models, Stable Diffusion, and FPDF. Development connects the individual generation functions into one workflow.

2. Environment Setup

Create and activate a Python virtual environment, install the project dependencies, and configure the Gemini API key and Hugging Face token as environment variables. The guide gives the example command ``uvicorn app.main:app --reload`` to start the local server.

3. Outline Generation

The `generate_outline()` function in `gemini_flash.py` uses Gemini Flash to create a structured five-panel outline. Each panel includes a panel number, title, scene description, and image-generation prompt.

4. Story Generation

The `generate_story()` function in `gemini_pro.py` expands the outline into a comic-style story with narration and character dialogue for each panel.

5. Image Generation

The `generate_image()` function in `image_generator.py` uses Stable Diffusion to create an illustration from a panel's image prompt. It sanitizes the prompt for safe filenames and saves the generated image in `static/panels`.

6. Layout Assembly

The `build_comic_layout()` function in `layout_builder.py` organizes the story and generated images, matching each image with its corresponding panel text. The resulting layout contains panel numbers, image paths, and text.

7. PDF Export

The `save_pdf()` function in `exporters.py` uses FPDF to compile the comic into a multi-page PDF. Panel images and narration are placed on consecutive pages, and the PDF is saved in `static/exports`.

8. FastAPI Integration

`routes.py` connects the functions. The `/generate` route processes form submissions; `/generate-comic/json` processes JSON requests and returns layout data and a PDF path; `/test-image` tests image generation independently. The homepage and export-success routes render their respective templates.

9. Frontend Development

The HTML, CSS, and Jinja2 interface includes `index.html` for input, `comic_preview.html` for panel-by-panel review, and `export_success.html` for export confirmation. Jinja2 loops through the layout to render the panel content dynamically.

10. Development Outcome

The development phase brings together input collection, AI outline and story generation, illustration creation, layout assembly, preview rendering, and PDF export in one application workflow.